ITU-MiniTwit - Group e - Report

Frederik Hørup , Marie Johansen , Nikolej Lundquist , Sara Bagger , Vitus Brodersen <>

May 13, 2026

Contents

1 Introduction

Authors:

2 System

2.1 Architecture and Design

Authors: Nikolej

Below is a diagram of the overall deployment architecture of the MiniTwit application. The system uses two load balancers and two production server instances each running all containerized services and both read and write to the same DigitalOcean Managed Database.

This setup is designed to prioritize availability. One can imagine the stakeholders of a social media platform pushing for availability, since every second of downtime is potential earnings lost.

The dual load balancer (LB) setup was chosen for this reason. The primary load balancer initially handles all traffic, but is replaced by the secondary in case of failure. This is enabled through the heartbeat messages exchanged between the two using VRRP. The secondary then takes ownership of the reserved IP and resumes operation. Essentially, one LB is active while the other is on standby, which keeps the system online and reduces one potential single point of failure.

The same principle is applied to the application layer, by using two MiniTwit production servers. Both run the exact same services and both are connected to the same database, which enables traffic to be served by either instance. This provides redundancy in case one server fails, allowing the team to bring it back up while the other instance continues to handle traffic.

However, there is one major drawback to this setup. The monitoring stack is duplicated as well. Logs and metrics for each server are separate, which compromises observability consistency. If one server goes down, that data is lost. This could be fixed by having a centralized monitoring stack e.g. on a separate device that all application nodes export to.

On the flip side, since both production serves run identical containerized environments, the system is easily reproducible and scalable.

While the monitoring and logging data is not centralized, the app database itself is. DigitalOcean provides a service for database hosting reducing the maintenance burden on the group. This allows focus on other aspects of the project rather than database administration, in exchange for some loss of control.

MiniTwit Deployment Diagram

2.2 Dependencies

Authors:

2.3 System States

Authors:

3 Process

3.1 CI/CD

Authors:

3.2 Monitoring

Authors:

3.3 Logging

Authors:

3.4 Security

Authors:

3.5 Availability and Scaling

Authors:

4 Reflection

Authors:

5 Use of Generative AI

Authors: Nikolej

During the development of this project we used ChatGPT in two main ways:

Debugging. Often when encountering unexpected bugs and error messages, we would consult ChatGPT about the cause and potential fixes While it could rarely fix the issues entirely by itself, more often than not, it pointed us in the right direction.

Research. This course presents challenges involving numerous technologies, most of which we were unfamiliar with. As such, ChatGPT was used to summarize lengthy documentation and provide concrete guides tailored to our situation.

Generating boilerplate / configurations. ChatGPT was also used for simple tasks like generating boilerplate code or writing Github Actions Workflow files. For example, the `build-report.yml` workflow, that converts the markdown source into a pdf.

The use of AI has made these tasks easier, spared us many frustrations and saved consiberable time during development. On the flip side, AI may have deprived us the extensive knowledge and experience you gain by, for example, painstaking reading of documentation or spending hours resolving a tiny bug.